JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING
P.O. Box 62000, 00200
Nairobi, Kenya
E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

This presentation is intended to covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green Exercise tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elearning@jkuat.ac.ke.

LESSON 7

Inheritance

Learning outcomes

Upon completing this topic you should be able to:

- Define inheritance and composition
- Implement inheritance and composition
- Differentiate derived class and base class
- Explain three types of inheritance: public, protected and private

7.1. Introduction to inheritance

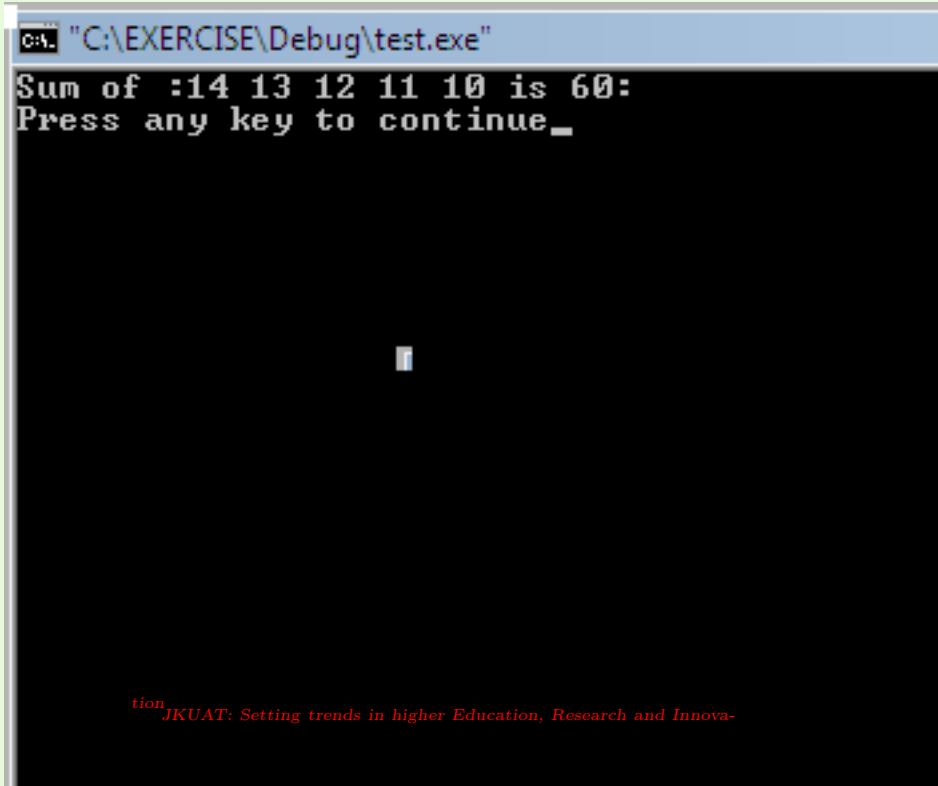
Inheritance is an "is-a" relationship. For example, a car is a vehicle.

In object-oriented programming, inheritance enables one to create a class from another class. The new class is referred to as a derived class, sub-class or a child class. The existing class is referred to as a base class, parent class or a super class. The derived class inherits properties of the base class. Inheritance can be either a single inheritance or a multiple inheritance. In single inheritance, the derived class has only one base class while in multiple inheritance the derived class has more than one base class.

In the above diagram, Vehicle is the base class. The classes car and lorry are derived from vehicle. Every car, every lorry is a vehicle.

If we have a class Vehice defined, to specify that class Car is derived from Vehicle class, we use the following syntax:

◀◀ ▶▶
◀ ▶
◀ Doc Doc ▶
Back Close



◀◀ ▶▶
◀ ▶
◀ Doc Doc ▶
Back Close

```
class Car:public Vehicle
{
.
.
.
};
```

Where class and public are keywords. public keyword is a member access specifier. It specify that all public members of the class Vehicle are derived as public members by the Car class. This means that public members of the class Vehicle become public members of the class Car.

Consider the following definition of Car class

```
class Car: private Vehicle
{
```

.

.

.

};

In this case, the public members of Vehicle class become the private members of the Car class.

The above definition of Car class is equivalent to the definition shown below.

class Car: Vehicle

{

.

.

.

};

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

The general format of defining a derived class is:

```
class className: memberAccessSpecifier baseClassName
{
.
.
.
};
```

where memberAccessSpecifier is public, protected or private. When no memberAccessSpecifier is indicated, it is assumed to be private.

Note

private members of base class can not be inherited by the derived class

public members of base class can be inherited by derived

class either as public or private members of the derived class

Derived class has additional members

Derived class can redefine public members of the base class

7.2. Redefining and Overloading Member Functions of the Base Class

Redefining a member function is having a member function in the derived class that has the same name and the same set of parameters as another member function in the base class. However, if the base class and derived class have members functions that have the same name but different sets of parameters, then this is fuunction over loading in the derived class.

7.3. Constructors of Derived and Base classes

Constructors initialize data members of a class. When a derived class is defined, it inherits the members of the base class, but it can not access private data members of the base class. When a derived class object is constructed, it must automatically execute one of the constructors of the base class. This means that the constructor of the derived class object must trigger execution of the constructor of the base class object. In that case, a call to the base class constructor is specified in the derived class constructor as shown below.

```
derivedClassName::derivedClassConstructor(parameter):
baseClassConstructor(arguments) {
.
.
```

```
.
}
e.g.
derivedClass::derivedClass(int x, int y, char w, int one):
baseClass(x, y) {
first=w;
second=one;
}
```

7.4. Virtual Functions and Pure Virtual Functions

A virtual function is a function that is bound with its call at run-time(also known as dynamic binding). The compiler does not generate the code to call a specific function. But instead it generate enough information to enable run-time system to

generate the code for appropriate function call.

virtual functions are only declared in the base class with the keyword virtual.

Example:

the calledclass that has been created or derived from another class can have a function that has been inherited from the based class having been redefined. Example

```
class Base{
public:
   virtual void print() {
      cout<<"This is base class"<<endl;
    }
};
```

A pure virtual function behaves just like a virtual function. However, it does not have the body. It is declared in a base class. A class that has a pure virtual function can not be instantiated. Such a class is also referred to as a abstract class. The general format of declaring a pure virtual function is shown below:

virtual return-type function-name(parameters)=0;

example:

virtual void print(void)=0;

A class that is derived from an abstract class must defined pure virtual functions in the base class. Otherwise the derived class will also be an abstract class.

Revision Questions

EXERCISE 1. ✍ Explain the difference between private and protected members of a class

Example ✎. Mark the following statements as true or false

a) The constructor of the derived class specify a call to the constructor of the base class in the heading of defining derived class constructor

b) Inheritance is "has a" relation

c) public members of a base class can be inherited either as public or private by the derived class

d) When initializing the objects of the derived class, the constructor of the base class is executed last

Solution: a) true b) false c) true d) false □

EXERCISE 2. ✍ Explain briefly why constructors are not inherited

EXERCISE 3. ✍ Differentiate overloading and redefining member functions

EXERCISE 4. ✍ Explain importance of inheritance

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

1.

Solutions to Exercises

Exercise 1. private members of a class can not be inherited by a derived class while protected members can be inherited by a derived class

Exercise 1